Music Application – Song ID Hashing

Implementation, execution, trace tables, complexity analysis and comparison of Hashing and Linear Search

1. Problem Statement

A music application stores the song IDs 105, 210, 315, 420, 525, 630, 735 and 840. A hash table is implemented using the Division Method. The assignment requires insertion, collision identification, searching using hashing and linear search, comparison of the number of operations/comparisons, load-factor calculation, complexity analysis and a final conclusion.

2. Assumption and Hash Table Design

For a clear collision demonstration, a hash table of size m = 10 is used. The Division Method is h(key) = key mod 10. Since all the given IDs end in 0 or 5, all of them initially produce the same hash value 5. Linear probing is used to resolve collisions: if a position is occupied, the next position is checked until an empty position is found.

Hash function: h(key) = key mod 10 **Collision resolution:** Linear probing **Table size:** 10

3. Insertion Trace Table

Step	Song ID	h(key)=key mod 10	Positions checked	Final position	Collision?
1	105	5	5	5	No
2	210	0	0	0	No
3	315	5	5, 6	6	Yes
4	420	0	0, 1	1	Yes
5	525	5	5, 6, 7	7	Yes
6	630	0	0, 1, 2	2	Yes
7	735	5	5, 6, 7, 8	8	Yes
8	840	0	0, 1, 2, 3	3	Yes

4. Hash Table After Each Insertion

After insertion	Index 0	1	2	3	4	5	6	7	8	9
105	–	–	–	–	–	105	–	–	–	–
210	210	–	–	–	–	105	–	–	–	–
315	210	–	–	–	–	105	315	–	–	–
420	210	420	–	–	–	105	315	–	–	–
525	210	420	–	–	–	105	315	525	–	–
630	210	420	630	–	–	105	315	525	–	–
735	210	420	630	–	–	105	315	525	735	–
840	210	420	630	840	–	105	315	525	735	–

5. Search Trace – Hashing

Song ID	Hash index	Positions checked	Comparisons
105	5	5	1
210	0	0	1
315	5	$5 \rightarrow 6$	2
420	0	$0 \rightarrow 1$	2
525	5	$5 \rightarrow 6 \rightarrow 7$	3
630	0	$0 \rightarrow 1 \rightarrow 2$	3
735	5	$5 \rightarrow 6 \rightarrow 7 \rightarrow 8$	4
840	0	$0 \rightarrow 1 \rightarrow 2 \rightarrow 3$	4

6. Search Trace – Linear Search

Song ID	Array position checked	Comparisons
105	1	1
210	$1 \rightarrow 2$	2
315	$1 \rightarrow 2 \rightarrow 3$	3
420	$1 \rightarrow 2 \rightarrow 3 \rightarrow 4$	4
525	$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5$	5
630	$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6$	6
735	$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7$	7
840	$1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8$	8

7. Performance Comparison

Song ID	Hashing comparisons	Linear-search comparisons
105	1	1
210	1	2
315	2	3
420	2	4
525	3	5
630	3	6
735	4	7
840	4	8
Total	**20**	**36**
Average	**2.50**	**4.50**

For the eight successful searches, hashing requires 20 comparisons in total (average 2.50), while linear search requires 36 comparisons in total (average 4.50). The hashing result is affected by collisions because several keys share the same initial hash index.

8. Load Factor and Collision Analysis

Load factor α = number of stored elements / table size = 8 / 10 = 0.8.

There are 7 collision events during insertion: every insertion after the first key that maps to index 5 or index 0 encounters an occupied position. Linear probing places the keys in consecutive positions. This creates clusters, so some searches require several probes instead of a single table lookup.

9. Theoretical Complexity

Method	Best case	Average case	Worst case	Extra space
Hashing with linear probing	O(1)	O(1)*	O(n)	O(m)
Linear Search	O(1)	O(n)	O(n)	O(1)

*Average-case O(1) for hashing assumes a suitable hash function, controlled load factor, and a good collision-resolution distribution. In the worst case, many collisions can make a hash search O(n). Here, the chosen hash function deliberately causes heavy collisions, so the observed search cost is higher than an ideal collision-free hash table.

10. C Program

```
#include <stdio.h>

#define SIZE 10
#define N 8

int hashTable[SIZE];
int songIDs[N] = {105, 210, 315, 420, 525, 630, 735, 840};

int hash(int key)
{
    return key % SIZE;
}

void initialize()
{
    int i;
    for (i = 0; i < SIZE; i++)
        hashTable[i] = -1;
}

void insert(int key)
{
    int index = hash(key);
    int start = index;

    while (hashTable[index] != -1)
    {
        index = (index + 1) % SIZE;

        if (index == start)
        {
            printf("Hash table is full.\n");
            return;
        }
    }

    hashTable[index] = key;
}

int hashSearch(int key, int *comparisons)
{
    int index = hash(key);
    int start = index;

    *comparisons = 0;

    while (hashTable[index] != -1)
    {
        (*comparisons)++;

        if (hashTable[index] == key)
            return index;

        index = (index + 1) % SIZE;

        if (index == start)
            break;
    }

    return -1;
}

int linearSearch(int key, int *comparisons)
{
    int i;
    *comparisons = 0;

    for (i = 0; i < N; i++)
    {
        (*comparisons)++;

        if (songIDs[i] == key)
            return i;
    }
```

```
    return -1;
}

void display()
{
    int i;
    printf("Index: ");
    for (i = 0; i < SIZE; i++)
        printf("%4d", i);

    printf("\nValue: ");
    for (i = 0; i < SIZE; i++)
    {
        if (hashTable[i] == -1)
            printf("%4s", "-");
        else
            printf("%4d", hashTable[i]);
    }
    printf("\n");
}

int main()
{
    int i, key, pos;
    int hashComp, linearComp;

    initialize();

    printf("HASH TABLE INSERTION\n");

    for (i = 0; i < N; i++)
    {
        key = songIDs[i];
        printf("\nInserting %d\n", key);

        if (hash(key) == 5 || hash(key) == 0)
            printf("Initial hash index = %d\n", hash(key));

        insert(key);
        display();
    }

    printf("\nSEARCH COMPARISON\n");
    printf("ID\tHashing\tLinear Search\n");

    for (i = 0; i < N; i++)
    {
        key = songIDs[i];

        pos = hashSearch(key, &hashComp);
        linearSearch(key, &linearComp);

        printf("%d\t%d\t%d\n", key, hashComp, linearComp);
    }

    printf("\nLoad factor = %d/%d = %.2f\n", N, SIZE,
           (float)N / SIZE);

    return 0;
}
```

11. Sample Execution Output

```
HASH TABLE INSERTION

Inserting 105
Index:    0   1   2   3   4   5   6   7   8   9
Value:    -   -   -   -   - 105   -   -   -   -

Inserting 210
Value:  210   -   -   -   - 105   -   -   -   -

Inserting 315
Value:  210   -   -   -   - 105 315   -   -   -

Inserting 420
Value:  210 420   -   -   - 105 315   -   -   -

Inserting 525
Value:  210 420   -   -   - 105 315 525   -   -

Inserting 630
Value:  210 420 630   -   - 105 315 525   -   -

Inserting 735
Value:  210 420 630   -   - 105 315 525 735   -

Inserting 840
Value:  210 420 630 840   - 105 315 525 735   -

SEARCH COMPARISON
ID      Hashing    Linear Search
105        1            1
210        1            2
315        2            3
420        2            4
525        3            5
630        3            6
735        4            7
840        4            8

Load factor = 8/10 = 0.80
```

12. Comparison and Discussion

The experiment shows that collision handling has a direct effect on the observed number of search comparisons. Although hashing is designed to provide average O(1) search, a poor or highly clustered hash distribution can increase the number of probes. Linear search does not depend on a hash function, but it may need to inspect many records before finding a requested ID. In this experiment, hashing uses fewer comparisons overall (20 versus 36) for the eight successful searches.

The load factor of 0.80 means that 80% of the table positions are occupied. As a table becomes more full, linear probing generally experiences more clustering and longer probe sequences. A larger table and/or a better hash function could reduce collisions and improve the observed performance.

13. Final Conclusion

For this small song-ID application, hashing with a suitable hash function and an appropriate table size provides fast average-case searching and, in this experiment, requires fewer comparisons than linear search. Therefore, hashing is an appropriate approach when the application needs frequent song-ID lookups. However, the experiment also demonstrates that the choice of hash function, table size, load factor and collision-resolution method is important. The chosen Division Method with table size 10 produces many collisions because of the pattern in the given IDs. Using a better-sized table or a better-distributing hash function would reduce clustering and improve performance.

14. GitHub Repository Contents

Folder/File	Content
src/hashing.c	C implementation of hash table, insertion, collision handling and hashing search
src/linear_search.c	C implementation of linear search
input/input.txt	Given song IDs
output/output.txt	Actual execution output
trace_table/trace_table.xlsx	Insertion and search trace tables
analysis/complexity_analysis.txt	Time and space complexity
analysis/comparison_table.txt	Hashing vs linear-search comparison
conclusion/final_conclusion.txt	Final conclusion
README.md	Assignment overview and repository description

Repository link: https://github.com/your-username/music-song-hashing-assignment

Note: Replace the placeholder repository link with your actual GitHub repository URL after uploading the files.